BitGo API Reference — Key Methods for Deposit Address Flow

Sourced from developers.bitgo.com — July 2, 2026

---

1. Initialize the SDK

```
const { BitGo } = require('bitgo');

const bitgo = new BitGo({
  accessToken: '<ACCESS_TOKEN>',
  env: 'test',       // 'test' → app.bitgo-test.com | 'prod' → app.bitg
});
```

For production with custom endpoint:

```
const bitgo = new BitGo({
  accessToken: '<ACCESS_TOKEN>',
  env: 'custom',
  customRootURI: 'https://app.bitgo.com',
});
```

---

2. Get a Wallet

Standard wallet lookup (all docs use this):

```
const wallet = await bitgo.coin('<ASSET_ID>').wallets().get({ id: '<WAL
```

Examples from docs:

```
// BTC testnet
const wallet = await bitgo.coin('tbtc4').wallets().get({ id: walletId }

// Generic custody
const wallet = await bitgo.coin('<ASSET_ID>').wallets().get({ id: '<DEP

// Go Account (off-chain)
const wallet = await bitgo.coin('ofc').wallets().get({ id: walletId });
```

‼️ **Note**: The docs use `.wallets().get()` — **not** `.wallets().getWallet()`.

---

3. Create a Receive Address

Basic:

```
const address = await wallet.createAddress();
```

With label:

```
const address = await wallet.createAddress({ label: 'Customer ABC' });
```

Go Account with token:

```
const address = await wallet.createAddress({
  onToken: '<OFF-CHAIN_ASSET_ID>'  // e.g., 'ofctsol' for Solana
});
```

Response shape:

```
{
  "id": "631283e10e052800066295e210da142a",
  "address": "2N9wCEV3KGEFsyo9xoUGjVYaSVwjSueutjz",
  "chain": 10,
  "index": 2,
  "coin": "tbtc4",
  "wallet": "6312824bf3281c0006fedaad1d667e67",
  "coinSpecific": {
    "redeemScript": "00200fda..."
  }
}
```

REST API equivalent:

```
POST /api/v2/{coin}/wallet/{walletId}/address
```

Note: Some networks like Ethereum don't immediately return a new multisig address — creating a new address requires a blockchain transaction. The `pendingChainInitialization` parameter identifies if an address is awaiting confirmation.

---

4. List Addresses

```
const wallet = await bitgo.coin('<ASSET_ID>').wallets().get({ id: '<WAL
const addresses = await wallet.addresses();
```

Response:

```
{
  "coin": "btc",
  "totalAddressCount": 5,
  "addresses": [
    {
      "id": "631283e10e052800066295e210da142a",
      "address": "bc1q8w3mcwt83tpmmr4reas3xt8t7rcshu7gztszew",
      "chain": 10,
      "index": 0,
      "coin": "btc"
    }
  ]
}
```

REST API:

```
GET /api/v2/{coin}/wallet/{walletId}/addresses
```

---

5. Webhook Registration

```
wallet.addWebhook({
  type: 'transfer',
  url: 'http://your.server.com/webhook',
  label: '<webhook_label>',
  numConfirmations: 6,
  allToken: false,
  listenToFailureStates: true
});
```

Response:

```
{
  "id": "6853113bd6d99d1109391bc98a0dbfd5",
  "label": "my-btc-transfer-webhook",
  "created": "2025-06-18T19:19:23.127Z",
  "scope": "wallet",
  "walletId": "67536a92b294f87c998ea39f85a6bdc7",
  "coin": "tbtc4",
  "type": "transfer"
}
```

Related API references:

- `POST /api/v2/{coin}/wallet/{walletId}/webhooks` — Add Wallet Webhook
- `POST /api/v2/webhook/secret` — Create Webhook Secret
- `POST /api/v2/{coin}/wallet/{walletId}/webhooks/simulate` — Simulate Webhook
- `POST /api/v2/webhook/verify` — Verify Webhook Notification (HMAC)

---

6. Create Wallet (Go Account)

```
const goAccount = await bitgo.coin('ofc').wallets().generateWallet({
  label: '<WALLET_NAME>',
  passphrase: '<SERVICE_USER_LOGIN_PASSPHRASE>',
  enterprise: '<CHILD_ENTERPRISE_ID>',
  type: 'trading',  // Go Accounts are trading type wallets
  passcodeEncryptionCode: '<ENCRYPTION_CODE>'
});
```

---

7. Coin & Token Identifiers

Network	Base Coin	Token	Use Case
Ethereum Holesky (testnet)	`hteth`	`hteth:tusdc`	ERC-20 USDC on testnet